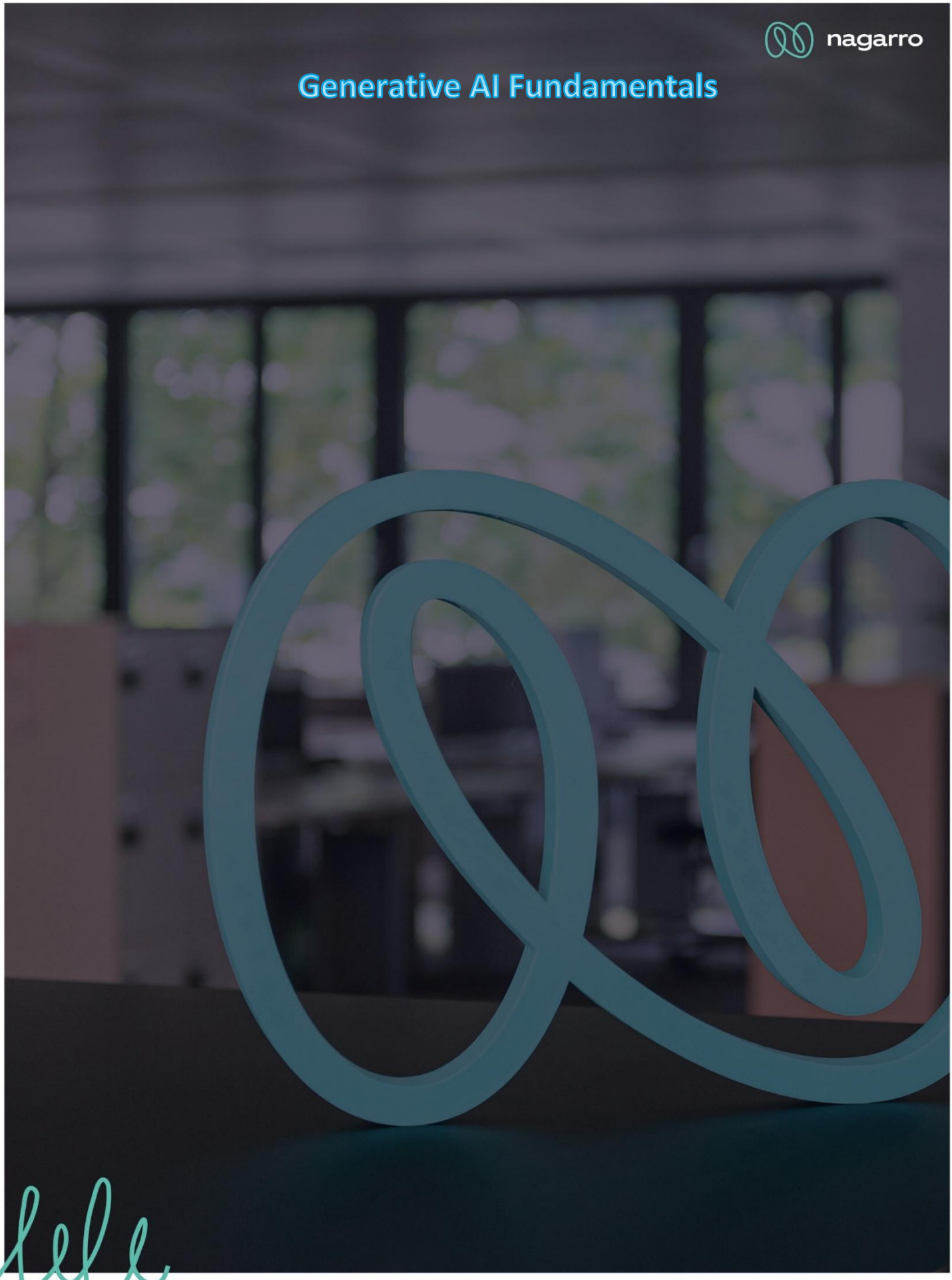
nagarro

Generative AI Fundamentals



Please Note:

This material is designed to be simple and easy to understand, making it suitable even for colleagues without a technical background. While the training program is entirely self-paced, if you encounter any difficulty in understanding the content, we recommend referring to the additional resources provided at the end to support your learning.

The training plan includes four short, week-wise exercises to be completed alongside the learning material. These exercises are meant for self-practice only and do not require submission. After completing the material and all 4 exercises, you will be required to work on a final assignment for evaluation purposes.

Introduction

Generative AI (Gen AI) refers to artificial intelligence systems that can generate new, original content, such as text, images, or even code, based on patterns learned from large datasets. Unlike traditional AI, which typically focuses on classification or prediction, **Generative AI** creates content that mimics the structure and style of the data it was trained on.

In this training material, Gen AI is specifically highlighted in relation to **Large Language Models (LLMs)**, which are a subset of Generative AI. These models, like GPT (Generative Pretrained Transformer), can understand and generate human-like text based on the input they receive. The material also includes applications of Generative AI, such as building Q&A apps, text generation applications, and image generation applications, using tools and APIs like **OpenAI's GPT**, **DALL-E**, and **Hugging Face**.

In essence, **Generative AI** in this context is about using machine learning models to **create new content**—whether that's generating text, creating images, or developing interactive AI applications—through the use of advanced models like LLMs and related technologies.

Key Takeaway:

By the end of this pathway, you'll will understand **Generative AI** and **LLMs**, and also the practical ability to **develop real-world applications**. Whether it's building chatbots, generating content, or integrating AI capabilities, you'll be ready to create AI-driven solutions.

nagarro

4-Week Learning Pathway for Generative AI and Large Language Models (LLMs)

This document provides a structured 4-week learning pathway combining open-source resources and hands-on exercises. It includes lessons from the Microsoft Generative AI for Beginners repository and other curated resources.

Week 1: Introduction to Generative AI and LLMs

Goal: Build foundational knowledge of Generative AI and understand how LLMs work.

- **Course Setup**
  - Learn how to set up your development environment.
- **Introduction to Generative AI and LLMs**
  - Understand what Generative AI is and how Large Language Models (LLMs) work.
- **Exploring and Comparing Different LLMs**
  - Learn how to select the right model for your use case.
- **Hands-On Exercise:** Experiment with OpenAI's GPT or another LLM to generate ideas, summaries, and more.

Goal: Master the fundamentals of prompt engineering.

- **Understanding Prompt Engineering Fundamentals**
  - Hands-on learning of prompt engineering best practices.
- **Creating Advanced Prompts**
  - Apply techniques like few-shot, zero-shot, and chain-of-thought prompting to improve outcomes.
- **Practical Exercise:** Design prompts for various tasks, such as data summarization and coding assistance.

Exercise 1: – Intro to LLMs and Prompt Engineering

1. Summarize a News Article
   a. Task: Take a long news article and generate a 50-word summary.
   b. Tools: ChatGPT, Claude AI, Perplexity AI
2. Generate a Short Story from a Prompt
   a. Task: Write a 100-word short story based on the prompt: *"A lost robot finds its way home."*
   b. Tools: Google Gemini, Hugging Face Chat, OpenAI Playground
3. Translate a Paragraph into Another Language
   a. Task: Translate a 5-sentence paragraph from English to French and back to English to check accuracy.
   b. Tools: DeepL, Google Translate, ChatGPT

4. Generate a Python Function
    a. Task: Generate a Python function that calculates the Fibonacci sequence up to N.
    b. Tools: Replit AI, ChatGPT, Google Gemini

5. Analyze Sentiment of a Customer Review
    a. Task: Input a customer review (e.g., *"The product arrived late, but the quality was great!"*) and determine if it's positive, negative, or neutral.
    b. Tools: Hugging Face Sentiment Analysis, MonkeyLearn, ChatGPT

Week 2: Building LLM Applications

Goal: Develop applications using Generative AI and LLM APIs.

- **Learn to run LLMs locally using Ollama and LM Studio for privacy and cost efficiency, and integrate them via Hugging Face APIs.**
    - **Practical Exercise: Build a Q&A app using Ollama or LM Studio locally, with Hugging Face API as a fallback for complex queries.**
    - **References: Ollama: ollama.ai, LM Studio: lmstudio.ai and Hugging Face API: huggingface.co/docs/api-inference**

- **Best Practices**
    - **Secure setups (restrict ports).**
    - **Optimize model size.**
    - **Handle API limits/errors.**

        - **Building Text Generation Applications**
            - Create a text generation app using Local LLM / Azure OpenAI or OpenAI API.
        - **Building Chat Applications**
            - Techniques for efficiently building and integrating chat applications.
        - **Practical Exercise:** Develop a chatbot using LangChain and OpenAI's API/ local llm.

Other Applications

Goal: Build search, image generation, and low-code AI applications.

- **Building Search Apps with Vector Databases**
    - Create a search app using embeddings and vector databases like Pinecone or FAISS.
- **Building Image Generation Applications**
    - Develop an image generation application using APIs like DALL-E.

Exercise: 2

Part 1: Chat Application using OpenAI Playground (Task 1: Build a Simple Chatbot)

- Use OpenAI Playground to create a chatbot that can answer user queries.
- Configure it to use gpt-3.5-turbo or gpt-4 with a system prompt like:
  "You are a helpful AI assistant that answers user questions politely and accurately."
- Experiment with different temperature and response lengths.
- Save sample conversations as a dataset for the next task.

Part 2: Create a Vector Database for Search (Task 2: Storing Chatbot Responses in a Vector Database)

- Convert chatbot responses into embeddings using OpenAI's text-embedding-ada-002.
- Store these embeddings in a vector database:
- Use FAISS (if running locally)
    - Implement a search function:
    - When a new user query is received, find the most similar past response using vector similarity search.
    - If a relevant response is found, retrieve it instead of generating a new response.

Tools & Resources : OpenAI Playground (https://platform.openai.com/playground)

Week 3-4: Integration, UX, and Security

Goal: Learn integration techniques, UX design, and security measures for Generative AI apps.

- **Integrating External Applications with Function Calling**
    - Understand function calling and its use cases for applications.
- **Securing Your Generative AI Applications**
    - Understand threats to AI systems and methods to secure them.
- Guardrails

Advanced Topics and Open-Source Tools

Goal: Dive into advanced frameworks, open-source models, and RAG.

- **Retrieval-Augmented Generation (RAG) and Vector Databases**
    - Build a RAG application using LangChain and vector databases.
- **Open Source Models and Hugging Face**
    - Create an application using open-source models available on Hugging Face.
- **AI Agents**
    - Build an application using an AI agent framework like LangChain/LangGraph/AutoGen

Optional Extra Learning Resources

- **LangChain Documentation**
- **Hugging Face Transformers**

Exercise-3

Task 1: Build a RAG Application Using LangChain and a Vector Database

Objective: Implement a basic Retrieval-Augmented Generation (RAG) system using LangChain and a vector database like ChromaDB or FAISS.

Instructions:

1. Install the necessary libraries: pip install langchain chromadb openai
2. Load a few sample text documents (e.g., Wikipedia articles, PDFs).
3. Embed the documents into a vector database.
4. Create a LangChain pipeline that retrieves relevant documents based on a user query and generates responses using an LLM (OpenAI API or Hugging Face model).
5. Test your system with a few sample queries.

Task 2: Create an Application Using an Open-Source Model from Hugging Face

Objective: Build a simple AI application using an open-source model from Hugging Face.

Instructions:

1. Explore Hugging Face's model hub (https://huggingface.co/models).
2. Select an open-source model for text generation, summarization, translation, or another NLP task.
3. Use the transformers library to load and interact with the model (pip install transformers).
4. Build a simple Python script or web app that takes user input and generates output using the model.

Task 3: Build an Application Using an AI Agent Framework Like Langchain, Langgraph, Autogen

Objective: Create a simple AI agent that can interact with users and make decisions based on inputs.

Instructions:

1. Install LangChain/autogen.
2. Create a multi-step AI agent that interacts with a user.
3. Define different tools the agent can use (e.g., search API, calculator, or knowledge retrieval from a document).
4. Implement basic decision-making logic.

nagarro

Additional Resources:

AI and Machine Learning

- Machine Learning and Deep Learning tutorial
- https://www.kaggle.com/learn
- Machine Learning Tutorial Python -1: What is Machine Learning?
- Machine Learning by Andrew Ng
- Deep Learning Specialization by Andrew Ng
- Deep Learning" by Ian Goodfellow, Yoshua Bengio, and Aaron Courville
- https://www.datacamp.com/blog/what-is-ai-quick-start-guide-for-beginners
- https://mark-riedl.medium.com/a-very-gentle-introduction-to-large-language-models-without-the-hype-5f67941fa59e
- YouTube: "What is AI?": Quick overview of Gen AI concepts.
- DataCamp: "What is AI?": Beginner-friendly guide.
- Kaggle - Learn Data Science: Interactive tutorials for data analysis.

NLP and LLMs

- Natural Language Processing (NLP) Tutorial with Python & NLTK
- Spacy vs NLTK: NLP Tutorial For Beginners In Python
- Natural Language Processing with Classification and Vector Spaces
- https://www.kaggle.com/code/eneszvo/ner-named-entity-recognition-tutorial
- https://www.datacamp.com/blog/what-is-tokenization
- https://stackoverflow.blog/2023/11/09/an-intuitive-introduction-to-text-embeddings/
- https://medium.com/@saschametzger/what-are-tokens-vectors-and-embeddings-how-do-you-create-them-e2a3e698e037
- https://www.datacamp.com/tutorial/introduction-to-text-embeddings-with-the-open-ai-api

- DataCamp - "Natural Language Processing Fundamentals": Covers text preprocessing and NLP tasks.
- Coursera - "Natural Language Processing": Introduces NLP concepts.
- Medium: "A Gentle Introduction to LLMs": Non-technical LLM overview.
- Stack Overflow Blog: "Text Embeddings": Explains embeddings intuitively.
- DataCamp: "Text Embeddings": Practical guide to embeddings.
- Kaggle: "NER Tutorial": Hands-on NER exercise.
- YouTube: "NLP Tutorial with NLTK": Video tutorial for NLP basics.

Generative AI and Language Models

- YouTube: "How Large Language Models Work": Visual explanation of LLMs.
- DeepLearning.AI: "Open-Source Models with Hugging Face": Hands-on course for Hugging Face.
- "Language Models are Few-Shot Learners": GPT-3 paper for understanding LLMs.
- "Attention Is All You Need": Transformer paper for foundational knowledge.
- Medium: "How Large Language Models Work": Non-technical overview.

Hope you have a productive and enriching learning journey. 😊